

API Desing con Wesley Beary

Introducción

Wesley Beary, cofundador de Anchor y ex-ingeniero en empresas como Salesforce y Heroku, se define a sí mismo como un "connoisseur" (catador) de APIs. En el podcast, aboga por abandonar la ilusión de lograr una arquitectura abstracta perfecta desde el primer día, apostando en su lugar por la empatía, la iteración y la resolución de problemas reales de los desarrolladores.

APIs, suposiciones y la experiencia del Usuario

En la ingeniería de software, los diseños teóricos trazados en una pizarra rara vez sobreviven al contacto con el mundo real. Por este motivo, Beary redefine irónicamente las siglas API como *"Assumptions Probably Incorrect"* (Las suposiciones probablemente sean incorrectas). Frente a la sobreingeniería de intentar prever cada caso límite, el autor recomienda un desarrollo ágil basado en lanzar prototipos rápidos que se enfrenten cuanto antes a los usuarios.

Uno de los mayores obstáculos en el diseño de interfaces es el sesgo cognitivo del propio arquitecto. Al estar "demasiado cerca del problema", los creadores asumen erróneamente que los usuarios poseen su mismo contexto técnico. Para romper esta burbuja, Beary propone importar el *"blind play test"* (prueba a ciegas) clásico de los juegos de mesa: entregar la documentación a un programador externo y observar cómo intenta resolver un problema en absoluto silencio. Intervenir para ayudarlo invalidaría la prueba, ya que simula el entorno de producción real donde el creador no estará presente para guiar al usuario.

Estas evaluaciones empíricas son fundamentales para auditar si el sistema respeta el **Principio de Menor Sorpresa**. La consistencia es el pilar de la usabilidad: si la interfaz establece que el comando de borrado es delete, no debe desconcertar al usuario exigiendo remove o destroy en módulos similares. Como sintetiza Beary respecto a la complejidad:

"Intentar crear algo que técnicamente sea lo suficientemente bueno para todos, a menudo termina siendo demasiado confuso y no tan bueno para nadie."

Problemas del uso exclusivo de CRUD en APIs

En el desarrollo de APIs, es común diseñarlas siguiendo el modelo CRUD (Create, Read, Update, Delete) para gestionar recursos. Sin embargo, este enfoque puede generar problemas cuando se utiliza como única forma de interacción.

Uno de los principales inconvenientes es que las operaciones, especialmente Update, tienden a sobrecargarse. En lugar de limitarse a modificar datos, pueden terminar ejecutando múltiples acciones dependiendo de los parámetros recibidos. Esto introduce complejidad y hace difícil predecir el comportamiento del sistema, generando situaciones donde la respuesta a "¿qué hace este endpoint?" es simplemente "depende". Esta ambigüedad afecta tanto a quienes implementan la API como a quienes la utilizan.

Como solución, se propone complementar el CRUD con endpoints específicos para acciones. De este modo, se separa claramente la gestión del estado de los recursos (CRUD) y las acciones que provocan comportamientos.

Este enfoque mejora la claridad del diseño, facilita la comprensión del sistema y evita la sobrecarga de responsabilidades en un único endpoint.

Diferencias entre APIs web y CLI

Las APIs web y las interfaces de línea de comandos (CLI) no siempre tienen los mismos objetivos ni necesidades. Mientras que las APIs web suelen diseñarse como interfaces generales para distintos usuarios, los CLI están pensados para facilitar tareas concretas, a menudo automatizando múltiples pasos.

Un patrón común es que un comando del CLI ejecute varias llamadas a la API por detrás. Esto puede generar problemas importantes como dificultad para manejar transacciones y riesgo de estados inconsistentes si una parte falla.

Solución: mover la lógica a la API

Para evitar estos problemas, se propone trasladar esa lógica compleja desde el CLI hacia la API mediante endpoints específicos, es decir empujar la “complejidad hacia abajo”, esto se haría en casos de uso muy comunes entre los clientes, pasando de tener varias llamadas a la API a solo una.

Esto permite mejor control transaccional, menos errores y APIs más claras y robustas.

El Dilema del Versionado: Usabilidad frente a Semántica

La gestión del cambio en las interfaces de programación es uno de los desafíos más costosos en el ciclo de vida del software. Wesley Beary argumenta que el versionado debe evitarse en la medida de lo posible, priorizando cambios que no rompan la compatibilidad (*non-breaking changes*).

Aunque técnicamente la versión debería gestionarse mediante los *headers* de HTTP como metainformación del recurso, la realidad operativa demuestra que esto incrementa la fricción para el desarrollador; por ello, la práctica predominante es incluir la versión directamente en la ruta de la URL (ej. /v1/), privilegiando la usabilidad.

Un modelo de estudio relevante es el de **Stripe**, que históricamente ha implementado versiones basadas en fechas. Bajo este esquema, un usuario queda vinculado a la versión vigente el día de su integración inicial, garantizando una estabilidad casi indefinida.

Sin embargo, el coste técnico interno de esta estrategia es masivo. Para mantener la retrocompatibilidad, el backend debe gestionar múltiples capas de traducción que transforman peticiones de versiones antiguas ($V1 \rightarrow V2 \rightarrow V3$) hasta adecuarlas al contexto del sistema actual. Esta arquitectura genera una deuda técnica considerable y una infraestructura de pruebas no trivial, lo que obliga a las organizaciones a planificar periodos de deprecación prolongados y a utilizar técnicas como los **"brownouts"** (cortes temporales de servicio) para alertar a los usuarios rezagados antes del cese definitivo de una versión.

El Patrón Strangler Fig

Para evolucionar infraestructuras heredadas hacia nuevos backends sin comprometer la integridad de la API, se propone el uso del **Patrón Strangler Fig**. Esta metodología consiste en ejecutar de forma simultánea el sistema antiguo y el nuevo, dirigiendo las llamadas a ambos y comparando los resultados en tiempo real para verificar la paridad de comportamiento. Solo una vez que el nuevo backend demuestra consistencia absoluta durante un periodo significativo, se procede a retirar el sistema original. Este enfoque permite una migración incremental que "estrangula" la funcionalidad del legado hasta su completa sustitución.

Abstracciones y el Riesgo del Mínimo Común Denominador

La evolución de las APIs a menudo busca la interoperabilidad mediante abstracciones, un recurso muy utilizado es emplear el patrón *Adapter* para unificar el acceso a las diversas implementaciones.

Al intentar abstraer servicios inherentemente distintos, se corre el riesgo de caer en el "**mínimo común denominador**", donde la interfaz genérica oculta las funcionalidades específicas que diferencian a cada proveedor; esto es especialmente cierto en el mercado actual, donde las empresas compiten mediante la diferenciación técnica, por lo que mantener una capa de abstracción útil se convierte en una batalla constante contra las especializaciones.

El Flujo de Trabajo "Contract-First" con OpenAPI

Finalmente, para optimizar el ciclo de diseño y desarrollo, Beary propone un flujo de trabajo centrado en la especificación **OpenAPI**. Este enfoque reemplaza el modelo tradicional de "implementar y luego documentar" hacia un modelo donde la especificación actúa como un contrato legible por máquinas.

La metodología ideal sigue este orden:

1. **Escritura de la Especificación (Spec):** Es la fase más económica y permite iterar sobre el diseño sin haber escrito código de producción.
2. **Mocking:** Utilizando herramientas que generan servidores de simulación basados en la especificación, se obtienen respuestas realistas que permiten validar el diseño de inmediato.
3. **Desarrollo del Cliente:** Escribir el cliente (ej. una CLI) antes que el servidor revela rápidamente omisiones de campos o fallos en la lógica de negocio.
4. **Implementación del Servidor:** Es el último paso, una vez asegurado que la lógica se construya sobre una base de requerimientos ya validada.

Este flujo permite "fallar rápido y barato", sincronizando dinámicamente la documentación con las pruebas automatizadas y eliminando el problema de la documentación desactualizada.

Referencias:

[SE Radio 679: Wesley Beary on API Design – Software Engineering Radio](#)

[Strangler Fig Pattern - Azure Architecture Center | Microsoft Learn](#)